

TAREA 14

PLAN DE PRUEBAS

Andrés Serrano Modrego

1. Definición del alcance de la calidad	3
1.1. Objetivo del plan de pruebas	3
1.2. Alcance funcional	3
1.2.1. Gestión de usuario y progreso	3
1.2.2. Sistema de cartas (Deckbuilder)	3
1.2.3. Sistema de combate	3
1.2.4. Sistema de enemigos	3
1.2.5. Sistema de logros y desbloques	3
1.2.6. Persistencia de datos (SQLite)	3
1.2.7. Interfaces de administración (CRUD)	4
1.2.8. Interfaz de usuario (UI)	4
1.3. Alcance no funcional (RNF críticos)	4
1.3.1. Rendimiento (fluidez en PC Windows)	4
1.3.2. Estabilidad y robustez	4
1.3.3. Persistencia fiable	4
1.3.4. Usabilidad	4
1.3.5. Compatibilidad (Windows)	4
1.3.6. Seguridad básica e integridad	4
2. Diseño de la estrategia de pruebas	5
2.1 Principios generales	5
2.2 Tipos de pruebas seleccionadas y justificación	5
2.2.1. Pruebas unitarias	5
2.2.2. Pruebas de integración	5
2.2.3. Pruebas de sistema	5
2.2.4. Pruebas de aceptación (usuario)	5
2.2.5. Pruebas de rendimiento	6
2.2.6. Pruebas de seguridad básica / integridad	6
2.3 Secuencia de ejecución propuesta	6
2.4 Entornos de prueba	6
2.5 Roles y responsabilidades	6

1. Definición del alcance de la calidad

1.1. Objetivo del plan de pruebas

El objetivo de este Plan Maestro de Pruebas es definir qué aspectos críticos se controlarán para garantizar la calidad del software antes de su entrega final. En esta fase del proyecto no se ejecutan pruebas; se establecen los procedimientos, tipos de prueba, responsables, entornos y secuencia de ejecución que se aplicarán durante el desarrollo y validación del producto.

1.2. Alcance funcional

1.2.1. Gestión de usuario y progreso

Creación/selección de perfil de jugador.
Guardado y carga de progreso.
Persistencia de estado de partida (si aplica).

1.2.2. Sistema de cartas (Deckbuilder)

Carga de cartas desde base de datos.
Construcción de mazo, robo, descarte y mano.
Aplicación de efectos, costes y condiciones.

1.2.3. Sistema de combate

Turnos, acciones disponibles, resolución de efectos.
Cálculo de daño, bloqueos, estados alterados y sus duraciones.
Finalización de combate, recompensas y transiciones.

1.2.4. Sistema de enemigos

Carga de enemigos desde base de datos.
Patrones de ataque/IA básica, escalado (si existe), estados.
Integridad de estadísticas (vida, daño, habilidades).

1.2.5. Sistema de logros y desbloques

Desbloqueo de cartas por logros/nivel.
Registro de logros conseguidos.
Visualización en UI y persistencia.

1.2.6. Persistencia de datos (SQLite)

Integridad referencial del esquema.
Lecturas/escrituras de progreso, logros, cartas desbloqueadas.
Manejo de errores (DB ausente, corrupción, datos incompletos).

1.2.7. Interfaces de administración (CRUD)

Alta/edición/baja de cartas y enemigos (según alcance).

Validación de campos y consistencia de datos introducidos.

1.2.8. Interfaz de usuario (UI)

Navegación entre pantallas principales.

Consistencia visual y usabilidad en PC.

Mensajes de error y confirmaciones.

1.3. Alcance no funcional (RNF críticos)

1.3.1. Rendimiento (fluidez en PC Windows)

El combate debe mantenerse fluido y sin bloqueos perceptibles.

Cargas de pantallas y datos sin pausas prolongadas.

1.3.2. Estabilidad y robustez

El juego no debe cerrarse inesperadamente ante entradas no previstas.

El sistema debe gestionar estados inconsistentes (por ejemplo, datos incompletos en DB) con control de errores.

1.3.3. Persistencia fiable

El progreso, logros y desbloques deben mantenerse entre sesiones.

Las operaciones con SQLite deben ser atómicas y consistentes.

1.3.4. Usabilidad

Navegación clara, botones accesibles, feedback inmediato de acciones.

Evitar ambigüedad en combate (qué carta se juega, sobre qué objetivo, coste y efecto resultante).

1.3.5. Compatibilidad (Windows)

Correcto funcionamiento en ejecución exportada para Windows.

Rutas de ficheros y acceso a BD consistentes en el entorno objetivo.

1.3.6. Seguridad básica e integridad

Evitar corrupción o pérdida de progreso ante cierres inesperados.

Validación mínima de entradas en pantallas de administración.

2. Diseño de la estrategia de pruebas

2.1 Principios generales

La estrategia se basa en una ejecución progresiva, desde componentes aislados hasta validación integral del sistema. Se prioriza:

- Detectar fallos temprano (unitarias).
- Verificar integración entre subsistemas (integración/sistema).
- Confirmar cumplimiento de expectativas (aceptación).
- Controlar rendimiento y estabilidad (rendimiento/robustez).

2.2 Tipos de pruebas seleccionadas y justificación

2.2.1. Pruebas unitarias

Objetivo: validar la lógica de componentes aislados.

Aplicación en el proyecto:

- Cálculo de daño/defensa.
- Reglas de coste y jugada de cartas.
- Aplicación y expiración de estados.

Justificación: reduce defectos lógicos en mecánicas núcleo (combate y cartas).

2.2.2. Pruebas de integración

Objetivo: comprobar que subsistemas interactúan correctamente.

Aplicación en el proyecto:

- Cartas en combate (aplicación de efecto y actualización de UI).
- Enemigos en combate (turnos, decisiones, estados).
- Combate y progreso/logros (registro correcto tras finalizar).

Justificación: el riesgo principal del proyecto está en la interacción entre lógica de combate, datos y UI.

2.2.3. Pruebas de sistema

Objetivo: validar flujos completos end-to-end.

Aplicación en el proyecto:

- Inicio → selección de perfil → combate → recompensa → guardado → salida → carga.

Justificación: asegura que la aplicación cumple los casos de uso completos.

2.2.4. Pruebas de aceptación (usuario)

Objetivo: confirmar que el producto cumple expectativas funcionales y de usabilidad.

Aplicación en el proyecto:

- Pruebas guiadas con checklist (combate entendible, navegación clara, feedback visible).

Justificación: valida que el juego es utilizable y comprensible para un usuario final en PC.

2.2.5. Pruebas de rendimiento

Objetivo: asegurar fluidez y tiempos de respuesta aceptables.

Aplicación en el proyecto:

- Combate prolongado con múltiples estados y cartas.
- Carga de biblioteca de cartas / pantallas con listas.

Justificación: un deckbuilder debe mantener respuesta inmediata para ser jugable.

2.2.6. Pruebas de seguridad básica / integridad

Objetivo: garantizar consistencia de datos y evitar pérdidas.

Aplicación en el proyecto:

- Guardados consecutivos, cierre forzado, reanudación.
- Validación de entradas en administración (evitar datos imposibles).

Justificación: la persistencia (SQLite) es parte crítica del valor del producto.

2.3 Secuencia de ejecución propuesta

La ejecución seguirá el siguiente orden lógico:

1. Unitarias: validar reglas internas (cartas/daño/estados).
2. Integración: validar cooperación entre módulos (cartas, combate, enemigos, DB).
3. Sistema: validar flujos completos de usuario (end-to-end).
4. Rendimiento: medir fluidez en escenarios de carga real.
5. Aceptación: validar experiencia final y requisitos de usabilidad.
6. Regresión (continua): repetir subconjuntos de pruebas tras correcciones o cambios relevantes.

2.4 Entornos de prueba

Se definen los siguientes entornos:

Entorno de desarrollo

- Godot Editor (ejecución local).
- Base de datos SQLite de desarrollo.

Entorno de pruebas

- Ejecutable exportado para Windows (build de prueba).
- Base de datos SQLite de pruebas (copia controlada con datos semilla).
- Objetivo: validar comportamiento real fuera del editor.

Datos de prueba

- Conjunto mínimo: cartas/enemigos básicos para comprobar flujos.
- Conjunto ampliado: múltiples cartas y enemigos para estrés funcional y rendimiento.

2.5 Roles y responsabilidades

Dado que el proyecto se desarrolla de forma individual, se definen los roles:

Responsable de pruebas técnicas (Desarrollador)

- Diseña y ejecuta pruebas unitarias/integración/sistema/rendimiento.

- Registra incidencias y verifica correcciones (regresión).

Responsable de validación de aceptación (Usuario final simulado)

- Ejecuta checklist de aceptación orientado a jugabilidad y comprensión.
- Reporta problemas de usabilidad, claridad de UI y coherencia.